UE24CS341A — SOFTWARE ENGINEERING

SOFTWARE REQUIREMENTS SPECIFICATION

Jackfruit File Compression Tool

Basic ZIP-Style Compression Utility using Canonical Huffman Coding

Version 0.1.0 | DRAFT | September 2026

Project	Jackfruit — Basic ZIP-Style File Compression Tool (Huffman Coding)
Team	Saatwik • Tanmayi • Yuvraj • Vibhav
Lead Author	Saatwik (Architecture Owner & Integrator)
Course	UE24CS341A — Software Engineering, PES University
Document Control	SRS-JACK-0.1.0 • Pending Team Sign-off

Revision History & Approvals

Revision History

Version	Date	Author	Change Summary	Approval
0.1.0	28-09-2026	Saatwik	Initial baseline mapped to Course SRS Template (Sections 1, 2, 4.1 UC-01, NFRs, 2 Use-Case Diagrams, RTM)	Pending Team Sign-off

Approvals

Role	Name	Signature / Email	Date
Overall Lead & Integrator	Saatwik	saatwik455@gmail.com	28-09-2026
Testing & Coverage Lead	Tanmayi	tanmayi.qa@jackfruit.local	11-10-2026
CI/CD Lead	Yuvraj	yuvraj.devops@jackfruit.local	11-10-2026
Security & Quality Lead	Vibhav	vibhav.sec@jackfruit.local	11-10-2026
Course Coordinator / Evaluator	Instructor	Faculty Review	Pending

Document Authorship Map

Section	Owner
1. Introduction, 2. Overall Description, 4.1 UC-01, 7. UML Diagrams, 8. RTM	Saatwik (Lead Author, Architecture Owner)
4.2 UC-02 (Decompression)	Tanmayi (Testing Lead)
4.3 UC-03 (Statistics)	Yuvraj (CI/CD Lead)
4.4 UC-04 (Validation), 5.1 Security Requirements	Vibhav (Security Lead)

Table of Contents

1. Introduction 4

2. Overall Description 5

3. External Interface Requirements 7

4. System Features (Detailed Functional Requirements) 8

    4.1 UC-01: File Compression (Owner: Saatwik) 8

    4.2 UC-02: File Decompression (Owner: Tanmayi) 8

    4.3 UC-03: Inspect Compression Statistics (Owner: Yuvraj) 8

    4.4 UC-04: Validate Archive Integrity (Owner: Vibhav) 10

5. Non-Functional Requirements 11

    5.1 Security Requirements 11

6. Quality Attributes & Acceptance Tests 13

7. System Models and Diagrams (UML Use-Case Diagrams) 14

    7.3–7.5 Use-Case Diagrams with Actors, «include» and «extend» 15

8. Requirements Traceability Matrix (RTM) 20

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Jackfruit File Compression Tool (v1.0). It defines functional requirements, measurable non-functional constraints, security requirements, and verification criteria in accordance with the course software engineering guidelines.

1.2 Scope

Jackfruit provides lossless single-file compression and decompression using canonical Huffman coding wrapped in a custom `.jack` container format. Key capabilities include:

- Generating compressed archives with embedded symbol dictionaries and 32-bit CRC checksums.
- Byte-for-byte exact restoration of original files from valid archives.
- Fast metadata inspection reporting file sizes, compression ratio, and storage savings.
- Integrity verification protecting users against corrupted, truncated, or tampered archives.
- A menu-driven interactive Command-Line Interface (CLI).

> **EXCLUSIONS**
> Multi-file archive bundling (tar/zip directory hierarchy), encrypted password-protected archives, and distributed client-server processing are outside the scope of Release 1.0.

These exclusions are deliberate scoping decisions rather than oversights: each would introduce a materially larger design surface (directory tree traversal and multi-entry indexing for bundling, key management and authenticated encryption for password protection, and a client-server protocol with its own security model for distributed processing) that is not required to demonstrate the core Software Engineering learning objectives of this course — Agile planning, modular architecture, CI/CD automation, and verification discipline. They are recorded here explicitly so that reviewers and the development team share the same understanding of what Release 1.0 does *not* attempt, and are revisited as "Future Work" candidates in Section 4.6 of the companion Software Architecture and Design Specification.

1.3 Audience

- **Course Instructors & Evaluators** — for evaluating requirements completeness and traceability.
- **Development Team** (Saatwik, Tanmayi, Yuvraj, Vibhav) — for implementing and integrating modules against defined contracts.
- **QA & Testing Lead** (Tanmayi) — for constructing automated test suites in GoogleTest and manual validation plans.

1.4 Definitions and Acronyms

Term	Definition
CLI	Command-Line Interface.
CRC	Cyclic Redundancy Check (specifically IEEE 802.3 32-bit CRC-32).
MSB	Most Significant Bit.
NFR	Non-Functional Requirement.
RTM	Requirements Traceability Matrix.
SAD	Software Architecture and Design Specification.
STP	Software Test Plan.
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.

2. Overall Description

2.1 Product Perspective

Jackfruit is an independent, self-contained desktop software utility implemented in standard C++17. It operates natively in the terminal without external runtime dependencies (no database engines, daemon processes, or third-party compression libraries such as zlib).

2.2 Major Product Functions

1. **Compress File** — reads an input file, computes symbol frequencies, constructs a Huffman prefix tree, encodes the bitstream, and generates a `.jack` archive.
2. **Decompress File** — rebuilds the Huffman tree from archive metadata, reads the encoded bitstream, and recreates the exact original file.
3. **Inspect Statistics** — inspects archive headers to extract and format metrics (uncompressed size, compressed size, compression ratio, space saved percentage) without decompressing the payload.
4. **Validate Integrity** — checks magic bytes, format version, payload boundary consistency, and computes CRC-32 checksums to detect corruption.
5. **Interactive Navigation** — provides an interactive text menu with prompts, progress feedback, and colour-coded status messages.

2.3 User Roles and Characteristics

Role	Characteristics
Standard User	Requires an intuitive, numbered terminal menu to compress, decompress, or inspect files without memorizing command-line flags.
System Evaluator / Grader	Expects reliable error handling on edge cases (empty files, corrupted archives), ≥80% automated unit test coverage, and clear requirement traceability.
Maintenance Developer	Relies on decoupled C++ modules, public header interfaces under `include/`, and clean documentation.

2.4 Operating Environment

- **Operating Systems:** Linux (Ubuntu 22.04 LTS / 24.04 LTS), macOS (Darwin ARM64 / x86_64).
- **Compilers:** Clang++ (≥10.0) or GCC/G++ (≥9.0) supporting ISO C++17.
- **Build System:** CMake (≥3.14).
- **CI Environment:** GitHub Actions runner (`ubuntu-latest`).

2.5 Constraints

- **Language Standard:** ISO C++17 is mandatory.
- **Zero Third-Party Compression Libraries:** all algorithms (frequency tables, tree construction, bit-level I/O, CRC-32) must be natively implemented.
- **Interactive UI Requirement:** menu-driven interaction is required by course guidelines.
- **Binary Portability:** multi-byte integers in `.jack` files must use Big-Endian (Network Byte Order).

2.6 Assumptions and Dependencies

- The evaluation environment provides a POSIX-compliant filesystem with normal read/write permission semantics; no network-mounted or permission-restricted filesystem behaviour is assumed.
- Input files are assumed to fit on local disk twice over (original plus archive) during compression, since Jackfruit does not compress in place.
- The grading environment provides GitHub Actions runners (`ubuntu-latest`) with outbound network access sufficient to fetch GoogleTest via CMake `FetchContent` and to report to SonarCloud.
- Team members have equivalent local toolchains (Clang++/GCC supporting C++17, CMake ≥3.14) so that builds are reproducible across the four development machines described in Section 2.4.

- No requirement in this document depends on a specific terminal emulator's ANSI colour support beyond the widely available 8/16-colour escape sequences; a monochrome fallback is acceptable degradation, not a defect.

3. External Interface Requirements

3.1 User Interfaces

The application provides an interactive terminal user interface:

```
================================================
  JACKFRUIT HUFFMAN COMPRESSION TOOL
================================================
  1. Compress File
  2. Decompress File
  3. Inspect Compression Statistics
  4. Validate Archive Integrity
  5. Exit
------------------------------------------------
Enter choice [1-5]: _
```

- Navigation is handled through single-key or numeric input.
- File paths are collected interactively via text prompts with default suggestions.
- Live progress is rendered for long-running operations.
- Every invalid input (bad menu choice, non-existent path, malformed archive) returns the user to a safe, known menu state rather than terminating the program, so a single mistake never costs the whole session.
- Status messages distinguish severity visually: informational text in the default terminal colour, successful completions in green, and errors in red, so a user scanning a long session transcript can locate failures at a glance even on a monochrome-degraded terminal where colour is unavailable (the wording of the message carries the same information redundantly).

This interactive-first design is a direct response to the course requirement that the final deliverable demonstrate a usable, navigable interface rather than a bare set of command-line flags; it is intentionally layered on top of, not instead of, the underlying library functions in `src/compression/`, `src/decompression/`, `src/archive/`, and `src/integrity/`, so that the same core logic remains independently unit-testable without going through the CLI.

3.2 Hardware Interfaces

Standard storage interfaces (HDD/SSD) accessible via standard OS filesystem system calls. No specialized hardware is required.

3.3 Software Interfaces

- **C++ Standard Library:** relies on `<iostream>`, `<fstream>`, `<filesystem>`, and standard containers.
- **GoogleTest Framework:** used for automated unit and integration tests.

3.4 Communications Interfaces

All operations are local file-based I/O. No network protocols or sockets are utilized.

4. System Features (Detailed Functional Requirements)

> **INSTRUCTOR RULE**
> At least 15 Functional Requirements across the overall project. Jackfruit specifies 16 total Functional Requirements (4 per module use case).

4.1 UC-01: File Compression (Owner: Saatwik)

> **USER STORY**
> As a standard user, I want to compress a file I choose from the interactive menu, so that it takes up less disk space while remaining fully recoverable later.

This use case is the foundation of the system: it owns the frequency analysis, canonical Huffman tree construction, and bit-level encoding that every other use case either consumes (statistics, validation) or inverts (decompression). It is implemented first in the sprint plan precisely because of this dependency position.

Req ID	Requirement (the system shall…)	Pri.	Acceptance Criteria / Test Ref	Dependencies
JACK-F-001	Prompt the user for an input file path, verify file existence and read permissions, and display an error message if invalid.	High	Non-existent paths display error and return to menu safely. *TC-COMP-01*	Depends on CLI shell (`src/cli/`)
JACK-F-002	Compute byte frequencies over the input file and construct a deterministic Huffman tree, resolving frequency ties by lowest byte value.	High	Identical inputs produce identical prefix trees. *TC-COMP-02*	Pure core algorithm (`src/compression/`)
JACK-F-003	Encode file bytes into variable-length binary prefix codes packed MSB-first into bytes using `BitWriter`, flushing with zero-padding.	High	Encoded bitstream matches Huffman code assignments. *TC-COMP-03*	Uses `BitWriter` (`src/io/`)
JACK-F-004	Call `ArchiveWriter` to write the complete `.jack` archive (magic bytes `JACK`, version 1, original size, CRC-32, symbol table, bit-count, payload).	High	Created archive strictly complies with format spec. *TC-COMP-04*	Integrates with Yuvraj's `ArchiveWriter`

4.2 UC-02: File Decompression (Owner: Tanmayi)

> **USER STORY**
> As a standard user, I want to restore a previously compressed `.jack` archive back to its exact original file, so that compression never risks permanent data loss.

Decompression is the mirror image of UC-01 and is where the round-trip guarantee is actually proven: a compressor that cannot be reliably inverted is worthless regardless of its compression ratio. This is why JACK-F-008 (byte-identical restoration) is treated as a team-wide milestone rather than a routine requirement.

Req ID	Requirement (the system shall…)	Pri.	Acceptance Criteria / Test Ref	Dependencies
JACK-F-005	Prompt for a `.jack` archive path, parse and validate the fixed header (magic bytes `JACK` and version `0x01`).	High	Valid header accepted; invalid header rejected cleanly. *TC-DECOMP-01*	Uses `ArchiveReader`
JACK-F-006	Reconstruct the exact Huffman decoding tree from the archive's stored symbol frequency metadata.	High	Reconstructed tree topology matches encoder tree. *TC-DECOMP-02*	Mirror of compression tree
JACK-F-007	Read the compressed payload bitstream MSB-first using `BitReader`, terminating exactly when `total_bits` are processed.	High	Zero trailing padding bits are decoded as data. *TC-DECOMP-03*	Uses `BitReader` (`src/io/`)
JACK-F-008	Write decoded bytes to the destination file and verify that the output matches the original file byte-for-byte.	High	Round-trip diff produces 0 differences on all test files. *TC-DECOMP-04*	End-to-end milestone

4.3 UC-03: Inspect Compression Statistics (Owner: Yuvraj)

> **USER STORY**
> As a user who has already compressed several files, I want to quickly see how effective the compression was, so that I can judge whether Jackfruit is worth using on similar files in future.

This use case deliberately avoids decompressing the payload — it reads only the fixed header and the tree metadata section, which is why JACK-F-009 specifies a tight ≤50 ms latency bound even on large archives. The underlying header-parsing logic is shared with, and must stay consistent with, the validation logic in UC-04.

Req ID	Requirement (the system shall…)	Pri.	Acceptance Criteria / Test Ref	Dependencies
JACK-F-009	Read and parse `.jack` archive metadata without reading or decompressing the full compressed payload.	**High**	Header inspection completes in ≤50 ms even for large files. *TC-STAT-01*	Fast I/O seek
JACK-F-010	Calculate uncompressed size, compressed size, compression ratio (original / compressed), and percentage space saved.	**High**	Math accurate to 2 decimal places. *TC-STAT-02*	Math calculation module
JACK-F-011	Format and render calculated metrics in a clean, aligned terminal table.	**Medium**	Table output properly aligned and legible. *TC-STAT-03*	Terminal formatting
JACK-F-012	Detect non-Jackfruit or corrupted archive files during inspection and report a human-readable error.	**High**	Graceful error message without exceptions or crashes. *TC-STAT-04*	Integrates with validator

4.3.1 Detailed Use-Case Specification (UC-03)

- **Primary Actor:** End User / System Auditor via Interactive CLI Menu (Option 3) or CLI flag `--stat`.
- **Preconditions:**
  1. The target file path is accessible in the local filesystem with read permissions.
- **Main Success Scenario:**
  1. User selects "Inspect Compression Statistics" and provides target archive path (e.g. `report.txt.jack`).
  2. `ArchiveReader` seeks to byte offset `0`, reads the fixed 18-byte header, verifies magic bytes `0x4A41434B`, and reads `symbol_count`.
  3. `StatisticsEngine` queries filesystem for total on-disk archive size (S_{comp}) and extracts stored uncompressed size (S_{orig}).
  4. `StatisticsEngine` executes ratio calculation:

$$\text{Compression Ratio} = \frac{S_{\text{orig}}}{S_{\text{comp}}}$$

$$\text{Space Saved (\%)} = \left(1.0 - \frac{S_{\text{comp}}}{S_{\text{orig}}}\right) \times 100.0\%$$

  5. The CLI prints a formatted metrics summary table to standard output within ≤ 50 ms.
- **Edge & Boundary Conditions:**
  - **Zero-byte original file:** If $S_{\text{orig}} = 0$, division by zero is prevented; ratio and space saved are reported as `N/A`.
  - **Negative savings:** If $S_{\text{comp}} > S_{\text{orig}}$ (expansion on high-entropy random data), negative space savings percentage is displayed explicitly (e.g. `-12.50%`) with ratio < 1.00.
- **Postconditions:**
  - Zero mutations to target archive file; memory footprint remains ≤ 4 MB.

4.4 UC-04: Validate Archive Integrity (Owner: Vibhav)

> **USER STORY**
> As a user who has received or stored a `.jack` archive for some time, I want to confirm it has not been corrupted or tampered with, so that I can trust a decompression attempt before relying on its output.

This use case is the system's primary defensive boundary against untrusted input: every structural check it performs (magic bytes, version, symbol-count bounds, payload-length consistency, CRC-32) runs *before* any decompression is attempted elsewhere in the system, which is why the Security Requirements in Section 5.1 are anchored to this use case.

Req ID	Requirement (the system shall…)	Pri.	Acceptance Criteria / Test Ref	Dependencies
JACK-F-013	Verify that archive initial bytes strictly match the 4-byte ASCII signature `JACK` and supported version `1`.	High	Non-matching signatures rejected immediately. *TC-VALID-01*	Header guard
JACK-F-014	Verify structural bounds of the archive: symbol count (0≤N≤256) and matching file size vs header calculations.	High	Truncated or malformed headers fail validation. *TC-VALID-02*	Buffer overflow guard
JACK-F-015	Compute IEEE 802.3 CRC-32 checksum of decompressed data and compare against the stored header checksum.	High	Modified payload bits trigger a checksum mismatch alert. *TC-VALID-03*	CRC-32 engine
JACK-F-016	Display an interactive validation report indicating archive status (`VALID`, `CORRUPT_HEADER`, `CHECKSUM_MISMATCH`, `TRUNCATED`).	High	Clear colour-coded report rendered to console. *TC-VALID-04*	CLI reporting

4.4.1 Detailed Use-Case Specification (UC-04)

- **Primary Actor:** End User / Automated Security Pipeline via CLI Menu (Option 4) or CLI flag `--validate`.
- **Preconditions:** Target archive file path provided.
- **Two-Stage Validation Pipeline:**
  1. **Stage 1: Structural Integrity Verification (Fast Fail):**
     - **Magic Signature Check:** Must match `0x4A 0x41 0x43 0x4B` (`JACK`). Fails with exit code `2` (`ERROR_INVALID_HEADER`).
     - **Version Compatibility Check:** Must equal `0x01`. Unsupported versions fail with exit code `2`.
     - **Symbol Count Bound Check:** Must satisfy 0 ≤ `symbol_count` ≤ 256. Fails immediately if exceeded, preventing heap corruption.
     - **Header Completeness Check:** File size on disk must be at least 18 + (9 × `symbol_count`) bytes.
  2. **Stage 2: Payload Cryptographic Integrity Verification:**
     - Stream through payload bitstream, decode symbol bytes in memory, and incrementally update CRC-32 checksum using IEEE 802.3 standard polynomial (`0xEDB88320`).
     - Compare computed 32-bit checksum with header stored checksum.
     - If computed == stored ⟹ Archive status: **VALID** (Exit code `0`).
     - If computed != stored ⟹ Archive status: **CHECKSUM_MISMATCH** (Exit code `3`).
- **Postconditions:**
  - Clean console report rendered indicating exact error category; temporary stream memory deallocated safely with zero leaks.

5. Non-Functional Requirements

> **INSTRUCTOR RULE**
> At least 5 measurable Non-Functional Requirements.

Req ID	Requirement	Category	Pri.	Acceptance Criteria / Measurement
JACK-NF-001	The compression engine shall achieve a throughput ≥5.0 MB/s on standard text files on modern x86_64/ARM64 desktop processors.	Performance	High	Measured via release binary timing on a 10 MB benchmark file. *TC-Perf-01*
JACK-NF-002	Peak memory consumption during compression or decompression shall not exceed 64 MB regardless of input file size.	Resource Efficiency	High	Validated using memory profiling (Valgrind Massif / Instruments). *TC-Perf-02*
JACK-NF-003	The software shall never produce undefined behaviour, unhandled exceptions, or segmentation faults on any malformed or corrupted file input.	Reliability	High	Zero crashes when tested against 100 randomly fuzzed inputs. *TC-Rel-01*
JACK-NF-004	Encoded `.jack` archives shall be fully portable and byte-identical across Linux (Ubuntu) and macOS architectures.	Portability	High	Archive created on macOS decompresses identically on Linux runner. *TC-Port-01*
JACK-NF-005	Interactive CLI menu navigation response time shall be ≤100 ms for all menu state transitions.	Usability	Medium	Response latency verified during manual usability walkthrough. *TC-UX-01*

Each NFR above maps to a specific class of risk identified during architecture review (see SAD Section 3.7 Risks & Mitigations): JACK-NF-001 and JACK-NF-002 guard against the large-file performance degradation risk (RISK-01), JACK-NF-003 guards against the malformed-input crash risk that motivates the entire Security Requirements section below, and JACK-NF-004 guards against the endianness mismatch risk (RISK-03) that is only detectable by actually exchanging archives across operating systems in CI.

5.1 Security Requirements

> **INSTRUCTOR RULE**
> At least 2 Security Objectives and at least 5 Security Requirements.

5.1.1 Security Objectives

1. **Archive Integrity Assurance** — ensure that any tampering, bit-rot, or inadvertent corruption of archive contents is reliably detected before output files are committed to disk.
2. **Robustness Against Malicious Input** — ensure that maliciously constructed or corrupted binary headers cannot cause buffer overflows, memory corruption, or infinite loops.
3. **Filesystem Safety** — prevent arbitrary file overwrite or directory escape through path sanitization.

5.1.2 Security Requirements Table

Req ID	Requirement (the system shall…)	Pri.	Acceptance Criteria / Test Ref
JACK-SR-001	Reject any archive file whose first 4 bytes do not strictly equal `0x4A 0x41 0x43 0x4B` (`JACK`), aborting parsing before allocating memory.	High	Non-JACK files rejected with exit code. *TC-SEC-01*
JACK-SR-002	Validate that the header `symbol_count` field is ≤256. If greater, terminate parsing immediately with `ERROR_INVALID_HEADER`.	High	Rejects manipulated count preventing heap overflow. *TC-SEC-02*
JACK-SR-003	Validate that `total_bits` does not exceed maximum possible payload size (8×payload_bytes).	High	Prevents bitstream read out-of-bounds. *TC-SEC-03*
JACK-SR-004	Verify computed CRC-32 checksum against header-stored checksum before declaring decompression successful.	High	Catches single-bit modifications in payload. *TC-SEC-04*
JACK-SR-005	Sanitize all user-entered output file paths to block directory traversal sequences (e.g. `../../etc/`).	High	Rejects unsafe path traversal inputs. *TC-SEC-05*

5.1.3 Security Hardening & Fuzzing Methodology **(Owner: Vibhav)**

To guarantee memory safety and compliance with `JACK-NF-003` and `JACK-SR-001..005`:

- **Sanitizer Pipeline:** All test binaries are compiled with `-fsanitize=address,undefined` on CI runners.
- **Fuzz Testing Vectors:** Automated test suite feeds corrupted test archives with:
  1. Truncated headers (1 to 17 bytes).
  2. Oversized `symbol_count` (`0xFFFF`).
  3. Desynchronized `total_bits` declarations.
  4. Random single-bit and multi-bit payload bit flips.
- **Path Sanitization Logic:** Output path resolvers strictly strip `../`, `./`, leading slashes, and control characters before triggering filesystem operations.

6. Quality Attributes & Acceptance Tests

6.1 Exit Criteria for Acceptance

1. All 16 Functional Requirements (JACK-F-001 through JACK-F-016) implemented and verified.
2. All 5 Security Requirements (JACK-SR-001 through JACK-SR-005) passing security validation.
3. Automated unit test coverage ≥80% line coverage on core modules (compression, decompression, archive, integrity, io).
4. Zero critical or blocker issues flagged in SonarCloud static analysis.
5. All 40 manual test cases (10 per member) executed with passing status.

6.2 Acceptance Test Suites

Suite	Coverage
Compression Test Suite	Tests frequency table accuracy, deterministic tree generation, and MSB bit-packing.
Decompression Round-Trip Suite	Verifies byte-identical restoration on empty, small, large, binary, and repetitive files.
Archive & Statistics Suite	Validates header parsing speed and ratio calculations.
Security & Integrity Suite	Verifies CRC-32 checking and rejection of corrupted/truncated archives.

6.3 Traceability Approach

Every functional, non-functional, and security requirement in this document is assigned a stable identifier (`JACK-F-xxx` , `JACK-NF-xxx` , `JACK-SR-xxx`) the moment it is first written, and that identifier is never reused or renumbered even if the requirement text is later revised — revisions are tracked instead through the Revision History table at the front of this document. Section 8 maps each identifier forward to the design element that implements it and the test case that verifies it, which is the mechanism the team uses, sprint by sprint, to answer the question "is this requirement actually done" with evidence rather than opinion.

7. System Models and Diagrams

> **INSTRUCTOR RULE**
> At least 2 UML Use-Case Diagrams.

7.1 UML Use-Case Diagram 1: Core Compression & Decompression Lifecycle

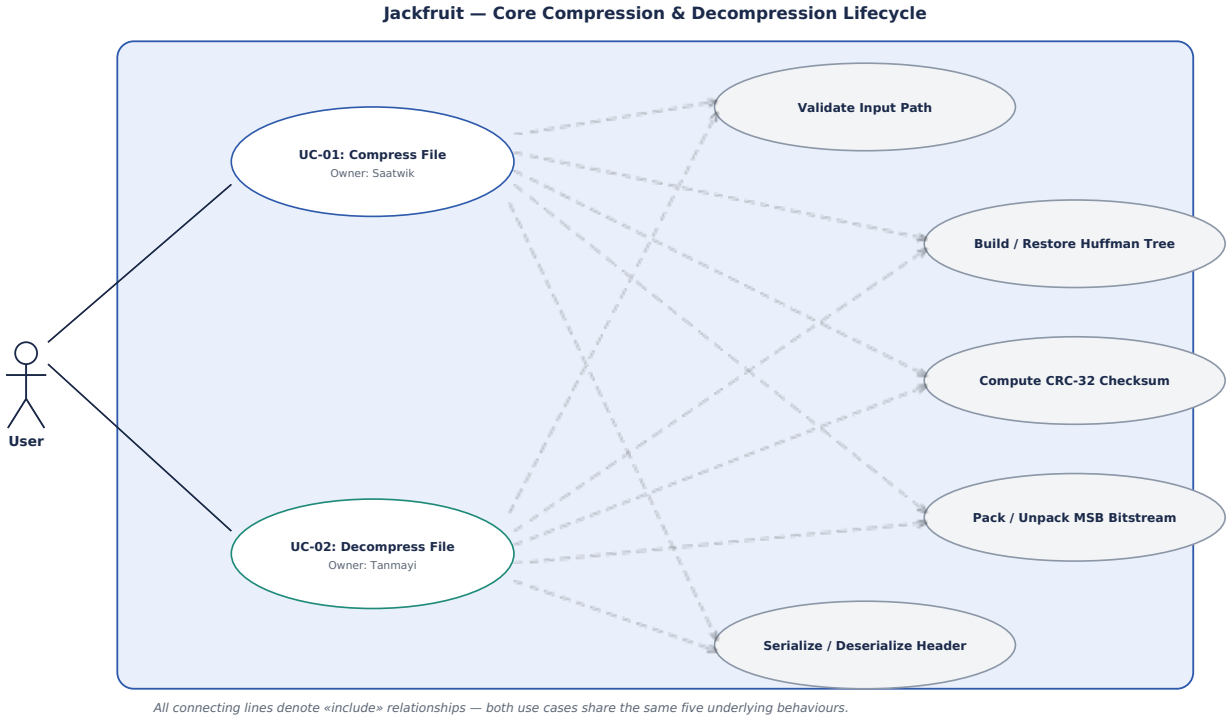


Fig. 2 — Use-Case Diagram 1: Core Operations

UC-01 and UC-02 are deliberately modelled as sharing the same five included behaviours (path validation, tree construction/reconstruction, CRC-32 computation, MSB bitstream packing/unpacking, and header serialization/deserialization) rather than as two unrelated use cases. This shared-behaviour structure is what makes the `src/io/` bit-ordering contract in the Archive Format Specification a single point of agreement between Saatwik and Tanmayi instead of two independently-guessed implementations.

7.2 UML Use-Case Diagram 2: Analytics & Integrity Verification Lifecycle

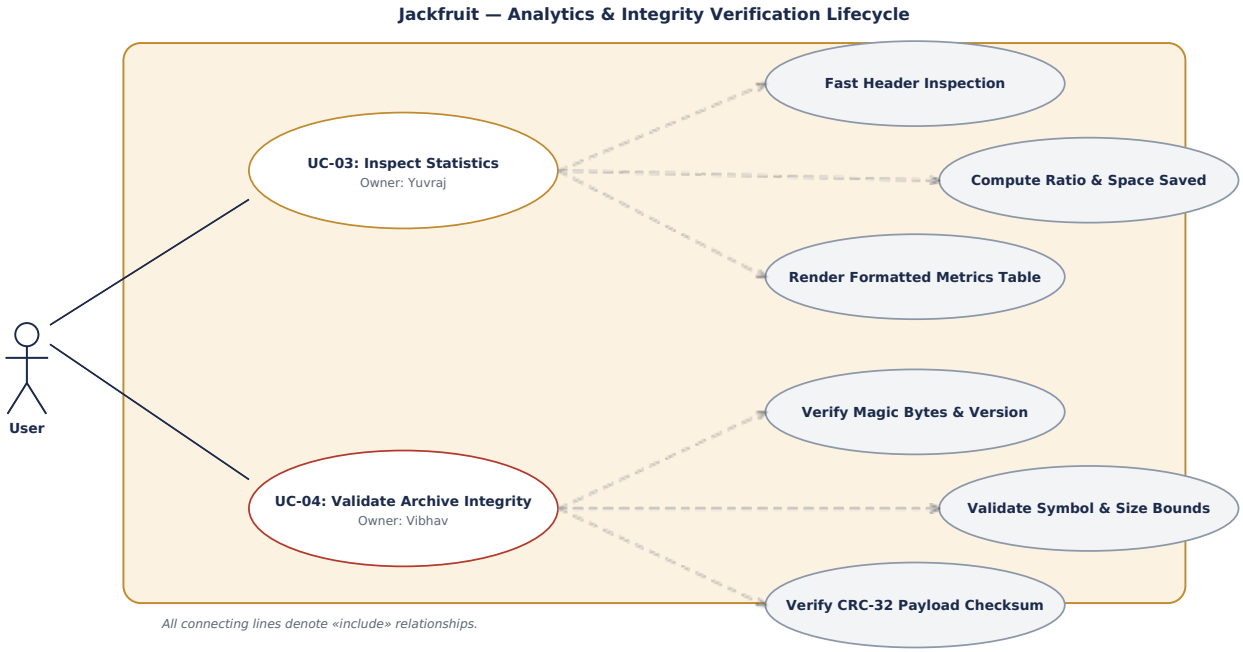


Fig. 3 — Use-Case Diagram 2: Analytics & Integrity

Unlike Diagram 1, UC-03 and UC-04 do *not* share included behaviours with each other — each owns its own three sub-behaviours. This reflects a real architectural boundary: statistics inspection (Yuvraj) is a read-only, best-effort reporting feature, while integrity validation (Vibhav) is a security-critical gate that other parts of the system are expected to call before trusting archive contents.

7.3 UML Use-Case Diagram 3: System Overview with Actors, «include» and «extend»

Diagrams 1 and 2 above show each use case together with the sub-behaviours it contains. Diagrams 3 to 5 present the same system in the conventional use-case notation: a system boundary, the actors outside it, **solid lines** for actor-to-use-case associations, and **dashed arrows** for the `«include»` and `«extend»` relationships. They introduce no new requirements; every use case (UC-01 to UC-04) and every behaviour shown is traceable to Section 4 or Section 5.1 of this document.

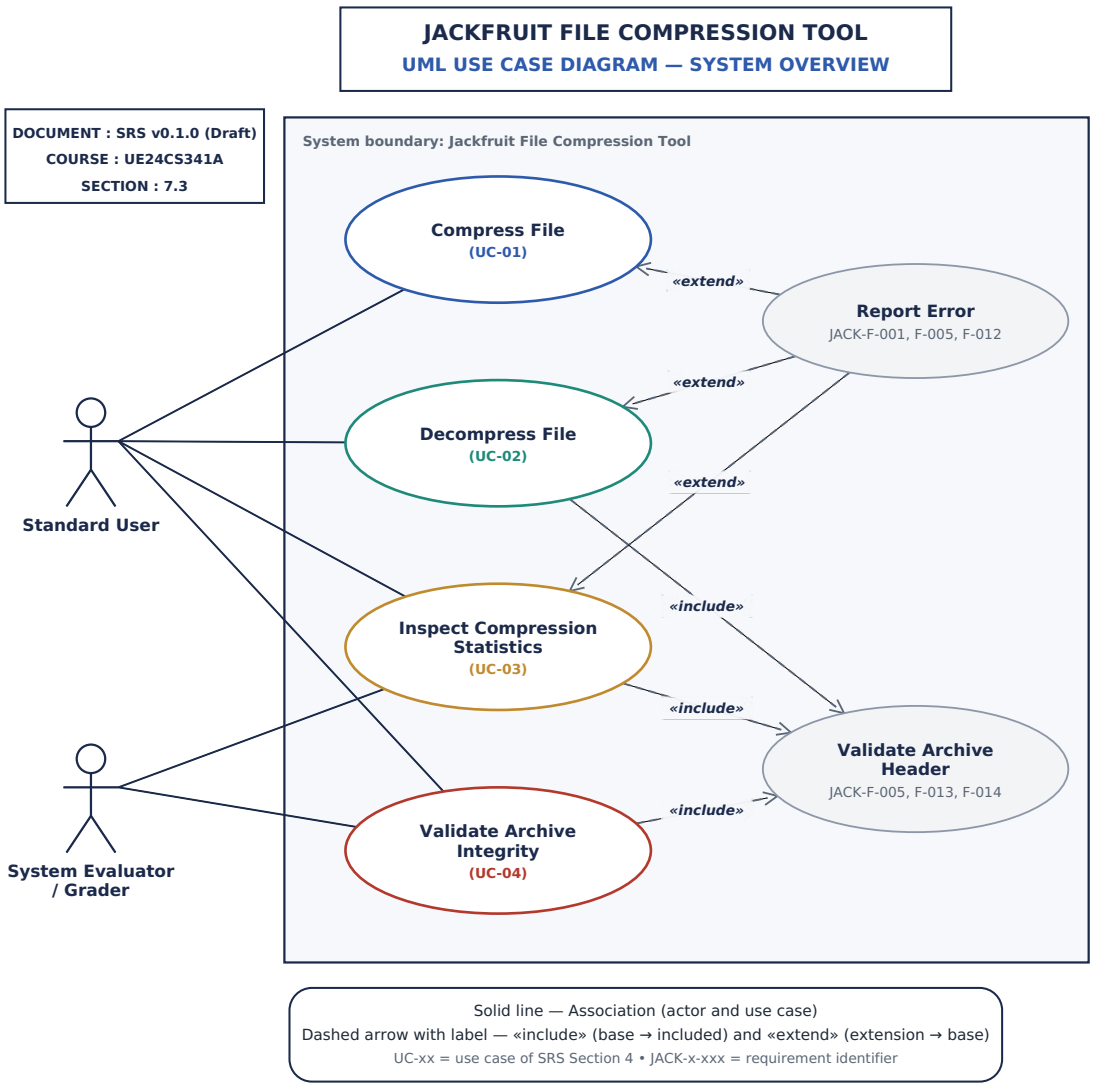


Fig. 4 — Use-Case Diagram 3: System overview with actors, «include» and «extend»

Actors

Actor	Type	Role (see Section 2.3)	Associated use cases
Standard User	Primary	Operates the numbered terminal menu to compress, decompress, inspect and validate files without memorising command-line flags.	UC-01, UC-02, UC-03, UC-04
System Evaluator / Grader	Primary	Exercises the tool against edge cases (empty files, corrupted archives), reads the statistics output and checks that tampering is detected.	UC-03, UC-04
File System	Secondary (external system)	Operating-system storage reached through `<fstream>` and `<filesystem>` (Sections 3.2 and 3.3). It is passive: it supplies and stores bytes but never starts a use case.	Shown in Diagrams 4 and 5 on the behaviours that read or write archive bytes

The Maintenance Developer role of Section 2.3 works with source code and documentation rather than through the running tool, so it is not an actor of these runtime use cases.

Reading the relationships

- **«include»** — the base use case *always* performs the included behaviour as part of completing normally. The arrow points from the base to the included behaviour.
- **«extend»** — the extending behaviour is added *only when its condition holds* (shown in square brackets where it is not obvious). The arrow points from the extension to the base it extends.

Diagram 3 deliberately shows only the two behaviours shared by three use cases each. **Validate Archive Header** is included by UC-02, UC-03 and UC-04, so a single implementation of the magic-byte, version and bounds checks (JACK-F-

005, JACK-F-012, JACK-F-013, JACK-F-014, JACK-SR-001, JACK-SR-002) guards every operation that opens an archive. **Report Error** extends UC-01, UC-02 and UC-03, and runs only when an input is invalid or an archive is corrupt (JACK-F-001, JACK-F-005, JACK-F-012); on a successful run it never appears.

7.4 UML Use-Case Diagram 4: Compression & Decompression in Detail (UC-01, UC-02)

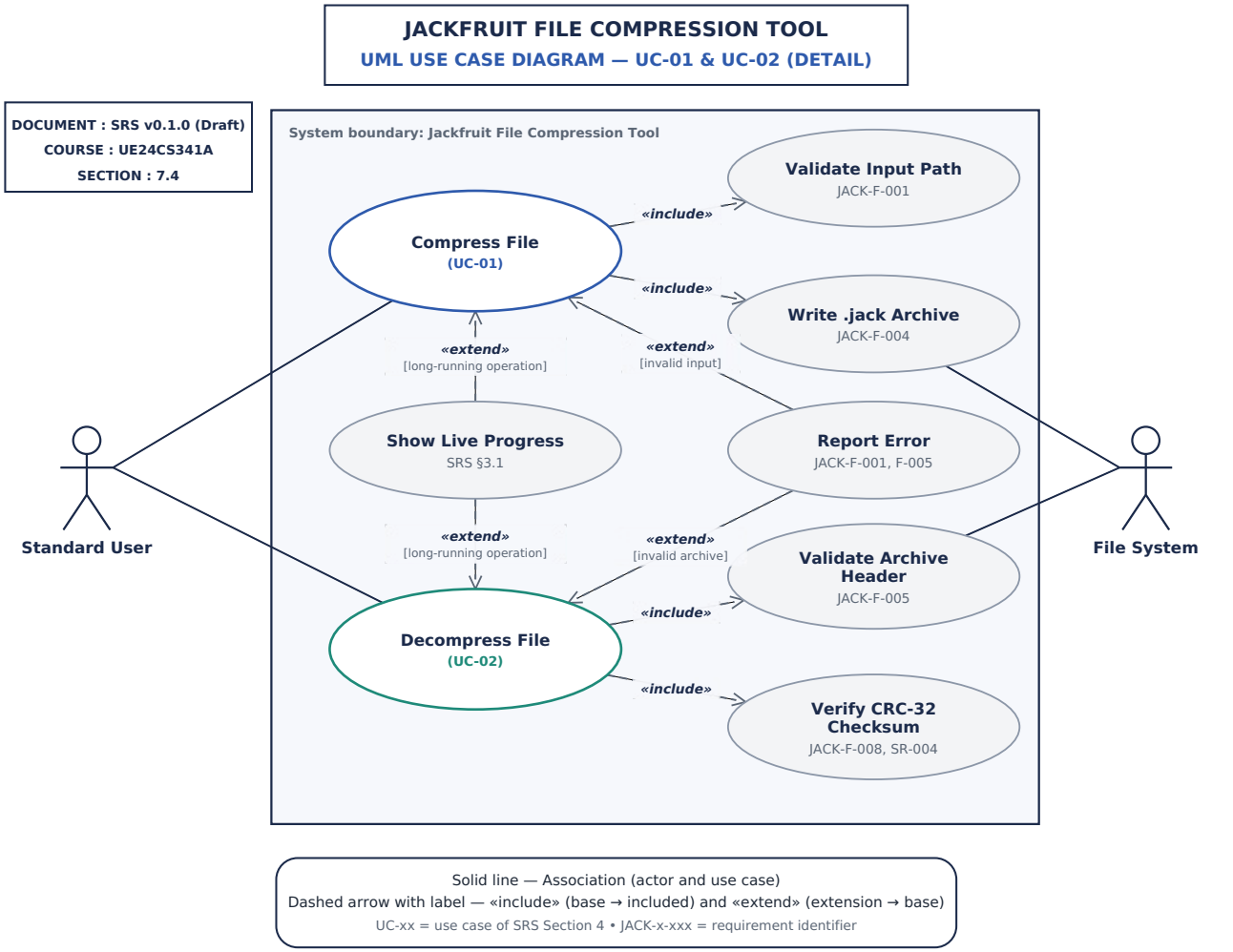


Fig. 5 — Use-Case Diagram 4: Compression and decompression in detail

- **UC-01 Compress File** always includes *Validate Input Path* (JACK-F-001) and *Write .jack Archive* (JACK-F-004).
- **UC-02 Decompress File** always includes *Validate Archive Header* (JACK-F-005) and *Verify CRC-32 Checksum* (JACK-F-008, JACK-SR-004), so a result is never declared successful before the checksum matches.
- **Show Live Progress** extends both use cases under the condition *long-running operation*, which is the behaviour stated in Section 3.1 (“live progress is rendered for long-running operations”).
- **Report Error** extends UC-01 when the input path is invalid or unreadable (JACK-F-001) and UC-02 when the archive is invalid or fails verification (JACK-F-005).
- The **File System** actor is associated with the behaviours that physically read or write archive bytes: *Write .jack Archive* and *Validate Archive Header*.

7.5 UML Use-Case Diagram 5: Statistics & Validation in Detail (UC-03, UC-04)

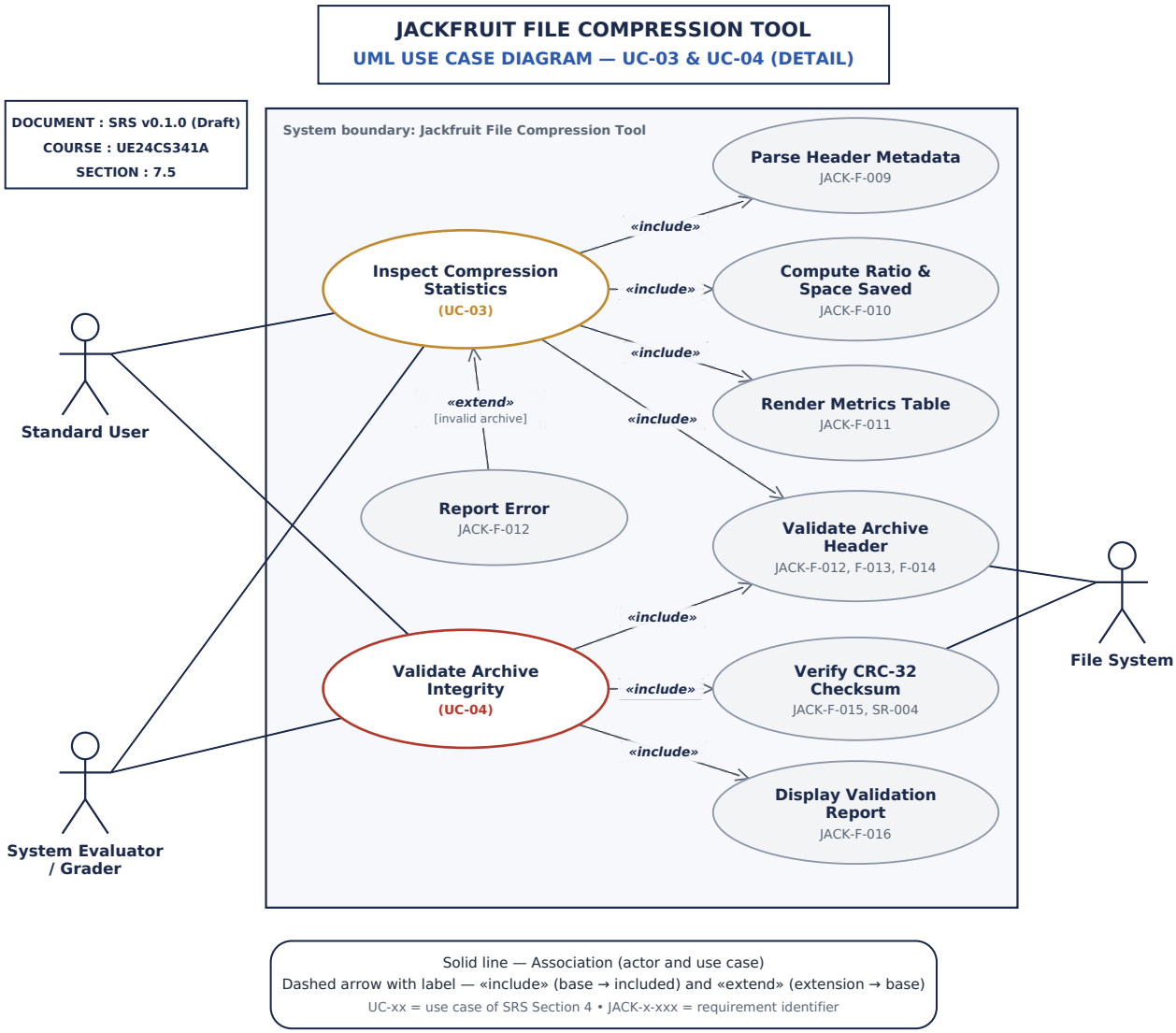


Fig. 6 — Use-Case Diagram 5: Statistics and validation in detail

- **UC-03 Inspect Compression Statistics** always includes *Parse Header Metadata* (JACK-F-009), *Compute Ratio & Space Saved* (JACK-F-010), *Render Metrics Table* (JACK-F-011) and *Validate Archive Header*, which is how JACK-F-012 detects non-Jackfruit or corrupted files before any figure is reported.
- **UC-04 Validate Archive Integrity** always includes *Validate Archive Header* (JACK-F-013, JACK-F-014), *Verify CRC-32 Checksum* (JACK-F-015, JACK-SR-004) and *Display Validation Report* (JACK-F-016).
- **Report Error** extends UC-03 when the archive is invalid (JACK-F-012). UC-04 has no extension: a corrupt, truncated or mismatching archive is its normal, reportable outcome (`CORRUPT_HEADER` , `CHECKSUM_MISMATCH` , `TRUNCATED`) and is handled by *Display Validation Report* itself.
- **Validate Archive Header** is the same shared behaviour that appears in Diagrams 3 and 4, drawn once here between the two use cases that rely on it.

Use-Case Relationship Catalogue

Use case	Actors	«include» (always)	«extend» (when condition holds)	Requirements
UC-01 Compress File (Saatwik)	Standard User; File System	Validate Input Path; Write .jack Archive	Show Live Progress [long-running operation]; Report Error [invalid input]	JACK-F-001 to F-004
UC-02 Decompress File (Tanmayi)	Standard User; File System	Validate Archive Header; Verify CRC-32 Checksum	Show Live Progress [long-running operation]; Report Error [invalid archive]	JACK-F-005 to F-008
UC-03 Inspect Compression Statistics (Yuvraj)	Standard User; System Evaluator / Grader; File System	Parse Header Metadata; Compute Ratio & Space Saved; Render Metrics Table; Validate Archive Header	Report Error [invalid archive]	JACK-F-009 to F-012
UC-04 Validate Archive Integrity (Vibhav)	Standard User; System Evaluator / Grader; File System	Validate Archive Header; Verify CRC-32 Checksum; Display Validation Report	— (failure outcomes are reported by the use case itself)	JACK-F-013 to F-016

Supporting Behaviour Traceability

Behaviour	Relation	Used by	Requirements	Implementing module
Validate Input Path	include	UC-01	JACK-F-001	src/cli, src/compression
Write .jack Archive	include	UC-01	JACK-F-004	src/archive
Validate Archive Header	include (shared)	UC-02, UC-03, UC-04	JACK-F-005, F-012, F-013, F-014, SR-001, SR-002	src/archive, src/integrity
Verify CRC-32 Checksum	include	UC-02, UC-04	JACK-F-008, F-015, SR-004	src/integrity
Parse Header Metadata	include	UC-03	JACK-F-009	src/archive
Compute Ratio & Space Saved	include	UC-03	JACK-F-010	src/archive
Render Metrics Table	include	UC-03	JACK-F-011	src/cli
Display Validation Report	include	UC-04	JACK-F-016	src/cli, src/integrity
Show Live Progress	extend	UC-01, UC-02	Section 3.1 (live progress for long-running operations)	src/cli
Report Error	extend	UC-01, UC-02, UC-03	JACK-F-001, F-005, F-012	src/cli

8. Requirements Traceability Matrix (RTM)

The table below is the authoritative cross-reference between every requirement in this document, the design artefact that addresses it (see the companion Software Architecture and Design Specification), the module that implements it, and the test case that verifies it. The **Status** column is updated at the end of every sprint review, never retroactively, so that the RTM's history (visible in the document's git log) doubles as an audit trail of when each requirement actually moved from Not Implemented to Approved.

Status Legend: **N** = Not Implemented (Sprint 1 Baseline) • **P** = In Progress • **A** = Approved / Verified.

Req ID	Requirement (short)	Design Spec Ref	Module	Test Case(s)	Status	Comments
JACK-F-001	Validate input file path	Sec 4.1 / DS-COMP-01	src/cli, src/compression	TC-COMP-01	N	Sprint 1 Spec
JACK-F-002	Frequency & tree building	Sec 4.1 / DS-COMP-02	src/compression	TC-COMP-02	N	Sprint 2 Target
JACK-F-003	MSB bitstream packing	Sec 4.1 / DS-COMP-03	src/io (BitWriter)	TC-COMP-03	N	Sprint 2 Target
JACK-F-004	Complete archive write	Sec 4.1 / DS-COMP-04	src/compression, src/archive	TC-COMP-04	N	Sprint 3 Milestone
JACK-F-005	Header parse & validate	Sec 4.2 / DS-DECOMP-01	src/archive (ArchiveReader)	TC-DECOMP-01	N	Sprint 2/3 Target
JACK-F-006	Tree reconstruction	Sec 4.2 / DS-DECOMP-02	src/decompression	TC-DECOMP-02	N	Sprint 3 Target
JACK-F-007	BitReader unpacking	Sec 4.2 / DS-DECOMP-03	src/io (BitReader)	TC-DECOMP-03	N	Sprint 2 Target
JACK-F-008	Byte-identical restoration	Sec 4.2 / DS-DECOMP-04	src/decompression	TC-DECOMP-04	N	Sprint 4 Team Milestone
JACK-F-009	Fast header inspection	Sec 4.3 / DS-STAT-01	src/archive	TC-STAT-01	N	Sprint 4 Target
JACK-F-010	Calculate compression stats	Sec 4.3 / DS-STAT-02	src/archive	TC-STAT-02	N	Sprint 4 Target
JACK-F-011	Format CLI statistics table	Sec 4.3 / DS-STAT-03	src/cli, src/archive	TC-STAT-03	N	Sprint 5 Milestone
JACK-F-012	Handle invalid archives	Sec 4.3 / DS-STAT-04	src/archive	TC-STAT-04	N	Sprint 5 Target
JACK-F-013	Verify signature & version	Sec 4.4 / DS-VALID-01	src/integrity	TC-VALID-01	N	Sprint 2 Target
JACK-F-014	Validate structural bounds	Sec 4.4 / DS-VALID-02	src/integrity	TC-VALID-02	N	Sprint 3 Target
JACK-F-015	CRC-32 integrity match	Sec 4.4 / DS-VALID-03	src/integrity	TC-VALID-03	N	Sprint 2 Target
JACK-F-016	Display validation report	Sec 4.4 / DS-VALID-04	src/cli, src/integrity	TC-VALID-04	N	Sprint 5 Milestone
JACK-NF-001	Throughput ≥5 MB/s	Sec 5.0 / DS-PERF-01	Core Pipeline	TC-Perf-01	N	Sprint 7 Benchmark
JACK-NF-002	Memory peak ≤64 MB	Sec 5.0 / DS-PERF-02	Core Pipeline	TC-Perf-02	N	Sprint 7 Benchmark
JACK-NF-003	Zero crashes on bad input	Sec 5.0 / DS-REL-01	All Modules	TC-Rel-01	N	Sprint 7 Fuzzing
JACK-NF-004	Cross-platform portability	Sec 5.0 / DS-PORT-01	I/O & Archive	TC-Port-01	N	CI Automated
JACK-NF-005	UI response ≤100 ms	Sec 5.0 / DS-UX-01	src/cli	TC-UX-01	N	Sprint 8 Verification
JACK-SR-001	Magic byte gating	Sec 5.1 / DS-SEC-01	src/integrity	TC-SEC-01	N	Sprint 2 Target
JACK-SR-002	Symbol count bound ≤256	Sec 5.1 / DS-SEC-02	src/integrity, src/archive	TC-SEC-02	N	Sprint 2 Target
JACK-SR-003	Payload bit bound check	Sec 5.1 / DS-SEC-03	src/io	TC-SEC-03	N	Sprint 2 Target
JACK-SR-004	CRC-32 mismatch rejection	Sec 5.1 / DS-SEC-04	src/integrity	TC-SEC-04	N	Sprint 2 Target
JACK-SR-005	Output path sanitization	Sec 5.1 / DS-SEC-05	src/cli	TC-SEC-05	N	Sprint 6 Security Pass

— End of Software Requirements Specification v0.1.0 —